

Project: Synchronous FIFO (First-In, First-Out) Buffer

RTL Design Specification: Synchronous FIFO

Version: 1.0

Status: Draft for Implementation

1. Functional Overview

The Synchronous FIFO is designed to store and forward 32-bit data words within a single clock domain. It acts as a buffer to handle temporary data bursts between a "Producer" and a "Consumer."

2. Interface Definitions (I/O Ports)

Signal Name	Dir	Width	Description
clk	I	1	Master Clock
rst_n	I	1	Asynchronous Reset (Active Low)
w_en	I	1	Write Enable: Push data into FIFO
w_data	I	32	Data input to be stored
r_en	I	1	Read Enable: Pop data from FIFO
r_data	O	32	Data output
full	O	1	High when FIFO is at max capacity
empty	O	1	High when FIFO has no data

3. Micro-Architecture & Data Path

- Storage Element: A dual-port distributed RAM or a 2D array of registers.
- Depth: 16 Words (configurable via parameter).
- Pointer Logic:
 - `wr_ptr`: Increments on `w_en` if `full` is low.
 - `rd_ptr`: Increments on `r_en` if `empty` is low.
- Status Logic: The `count` register tracks the number of items.
 - `full` is asserted when `count == 16`.
 - `empty` is asserted when `count == 0`.

4. Clock and Reset Strategy

- Clocking: All sequential logic triggers on the rising edge of `clk`.
- Reset: Upon `rst_n` falling edge:
 - `wr_ptr` and `rd_ptr` are cleared to 0.
 - `full` is de-asserted; `empty` is asserted.
 - `r_data` is driven to `32'h0`.

5. Timing Diagram (Handshake)

- Write Cycle: `w_data` must be stable at the rising edge of `clk` when `w_en` is high. `full` will update on the following clock cycle.
- Read Cycle: When `r_en` is high, `r_data` becomes available on the same cycle (First-Word Fall-Through) or next cycle (Standard). *Specified for this design: Standard (1-cycle latency).*

6. Design Constraints (PPA)

- Performance: Target frequency of 500 MHz on 7nm process.
- Area: Minimize area by using registers instead of SRAM for small depth (16 words).
- Power: Clock gating should be applied to the register array when `w_en` is inactive.

7. Verification Plan (Corner Cases)

1. Overflow: Attempt to write when `full` is high (Data must not be corrupted).

2. Underflow: Attempt to read when `empty` is high (Output must remain stable).
3. Simultaneous Access: Read and Write occurring on the same clock cycle.
4. Wrap-around: Ensure pointers correctly wrap from 15 back to 0.

Project: RTL Design Specification: UART Transmitter (TX)

1. Functional Overview

The UART TX module converts 8-bit parallel data from an internal bus into a serial bitstream. It automatically adds the required framing bits (Start, Stop) and manages the timing based on a configurable baud rate.

2. Interface Definitions (I/O Ports)

Signal Name	Dir	Width	Description
<code>clk</code>	I	1	System Clock (e.g., 50 MHz)
<code>rst_n</code>	I	1	Active Low Asynchronous Reset
<code>tx_start</code>	I	1	Pulse to initiate transmission
<code>data_in</code>	I	8	Parallel data byte to transmit
<code>baud_val</code>	I	16	Clock divider value for baud rate
<code>tx_serial</code>	O	1	Serial output pin (Idle high)
<code>tx_busy</code>	O	1	High during active transmission

3. Micro-Architecture & State Machine (FSM)

The design uses a 4-state Moore FSM:

1. IDLE: `tx_serial` is High. Wait for `tx_start`.
2. START: `tx_serial` goes Low for 1 baud period.

3. DATA: Shift out 8 bits of `data_in` (LSB first), 1 baud period per bit.
4. STOP: `tx_serial` goes High for 1 baud period to signal the end of frame.

4. Clock and Reset Strategy

- Baud Rate Generation: An internal counter increments every `clk`. When it reaches `baud_val`, a "baud_tick" is generated to trigger the next bit shift.
- Reset: Reset forces the FSM back to IDLE and sets `tx_serial` to 1.

5. Timing Diagram (Serial Protocol)

- Framing: 1 Start Bit (0), 8 Data Bits, 1 Stop Bit (1).
- Handshake: The `tx_busy` signal must be sampled by the internal logic. New data should only be sent when `tx_busy` is Low.

6. Design Constraints (PPA)

- Performance: Reliable operation up to 115,200 baud and higher.
- Area: Highly optimized combinational logic; minimal flip-flop usage (approx. 30–40 FFs).
- Power: Logic remains static (no switching) during the IDLE state.

7. Programming Model (Register Map)

If integrated into a CPU system:

- TX_DATA (Offset 0x0): Write 8-bit data here to start TX.
- TX_CTRL (Offset 0x4): Bit [0] enables/disables the transmitter.
- TX_STAT (Offset 0x8): Bit [0] is the `tx_busy` flag.

8. Verification Plan (Corner Cases)

1. Back-to-Back TX: Assert `tx_start` immediately after the Stop bit of the previous transaction.
2. Baud Rate Change: Change `baud_val` between transmissions to ensure the divider resets correctly.
3. Reset Mid-TX: Assert `rst_n` while a byte is half-transmitted; ensure `tx_serial` returns to Idle High immediately.

RTL Design Specification: I2C Master Controller

1. Functional Overview

The I2C Controller provides a standard 2-wire serial interface between the system and external I2C-compatible devices. It supports multi-master operation, clock stretching, and programmable bus speeds (Standard: 100kbps, Fast: 400kbps). [\[1, 2, 3\]](#)

2. Interface Definitions (Memory-Mapped Slave)

This module connects to the main system bus using an AXI4-Lite or APB interface.

Signal Name	Dir	Width	Description
S_AXI_ACLK	I	1	High-speed system clock
S_AXI_ARESETN	I	1	Active-low synchronous reset
I2C_SDA	IO	1	Serial Data line (Requires tri-state buffer)
I2C_SCL	IO	1	Serial Clock line (Requires tri-state buffer)
IRQ	O	1	Interrupt request for data-ready or error

3. Programming Model (Register Map)

Software interacts with the I2C hardware by reading and writing to these specific memory offsets.

Offset	Name	Reset Val	Description
0x00	PRER	16'hFF FF	Prescale Register: Sets the SCL frequency based on system clock.

0x02	CTR	8'h00	Control Register: Bit 7: Enable, Bit 6: Interrupt Enable.
0x04	TXR/RXR	8'h00	Data Register: Written data is transmitted; read data is received.
0x06	CR/SR	8'h00	Command/Status: Bits for Start, Stop, Read, Write, and Busy status.

4. Micro-Architecture: Bit Controller & Byte Controller

The design is partitioned into two main hierarchy levels:

- Bit Controller: Handles the low-level physical signal timing, including Start/Stop detection, arbitration, and clock synchronization.
- Byte Controller: Orchestrates bit sequences to form complete bytes, manages the acknowledge (ACK/NACK) bits, and handles the state transitions between IDLE, ADDRESS, and DATA.

5. Design Constraints (PPA)

- Timing: The [Setup and Hold times](#) for SDA relative to SCL must comply with Philips/NXP I2C standards.
- Performance: Logic must support "Wait State" generation if the slave device stretches the clock.
- Power: Static [Power Domains](#) should be implemented so the I2C block can be powered down when not in use. [

6. Verification & Testability

- Arbitration Test: Simulate two masters attempting to seize the bus simultaneously to verify that the "Arbitration Lost" interrupt triggers correctly.
- Bus Timeout: Verify the controller releases the bus if a transaction hangs longer than a specified limit.